

Relatório

trabalho prático

Redes de computadores



Universidade: Colégio Luís António Verney (Universidade de Évora)

Curso: Engenharia Informática

Identificações: Equipa E05

(Diogo Tavares Nº 58049 ,Francisco Rodrigues Nº 59119; David Ramalho Nº57823; Rafael Crihana Nº 58556)

Introdução

O projeto desenvolvido consiste num sistema de comunicação em tempo real baseado em UDP, onde os utilizadores podem interagir através de um sistema letra-a-letra.

Os principais módulos são o cliente e o servidor, comunicando através de datagramas UDP, com algumas funcionalidades complementares baseadas em TCP para a transferência de ficheiros.

A arquitetura foi desenhada para suportar:

- Vários clientes em simultâneo.
- O cliente envia mensagens ao servidor que as reenvia para os outros clientes.
- Os clientes podem receber mensagens do servidor, que podem ser mensagens de outros clientes, ou então notificações do servidor.
- Autenticação de contas.
- O servidor envia mensagens avisando se algum cliente entrou ou saiu do servidor.
- Criação e gestão de grupos.
- Envio de mensagens em tempo real (letra a letra).
- Troca de ficheiros entre utilizadores.
- Controlo de inatividade e timeout.

Mudanças do Protocolo para o projeto final

Estrutura das mensagens: Durante a realização do protocolo pensamos em diversas maneiras de enviar os dados necessário para as implementações futuras e chegamos à conclusão de que deveríamos utilizar mensagens no formato JSON da seguinte forma:

```
{"type": "tipo_de_mensagem",  
  "from": "utilizador_origem",  
  "to": "destino" (opcional),  
  "data": "conteúdo" (dependendo do tipo)}
```

Durante a discussão do protocolo, o professor sugeriu implementarmos uma struct simples com os dados necessários e da mesma maneira que faríamos em JSON e como um grupo decidimos que seria a melhor opção.

O formato final ficou com o seguinte formato:

```
typedef struct  
{  
    uint8_t type;           // 1: char, 2: comando (/)  
    char from[50];          // Nome do cliente  
    char data[BUFFER_SIZE]; // Dados da mensagem  
    char to[20];            // Destinatário  
    char group[20];         // Nome do grupo  
} Message;
```

Mantendo a maior parte do plano base, conseguimos implementar a ideia com o qual concordamos inicialmente.

O resto do protocolo fala do comportamento do servidor e em como deveríamos permitir variáveis para vários

Autenticação

- **Protocolo Inicial:** Não detalhava autenticação.
- **Projeto Final:** Implementado via mensagens MSG_AUTH (login) e MSG_CREATE (criação de conta), com dados armazenados em `contas.txt`.
 - **Limitação:** Senhas sem hashing (pouco seguro).

Grupos

- **Protocolo Inicial:** Mencionava extensões futuras como `group_create` e `group_msg`.
- **Projeto Final:** Implementado com comandos como:
 - `/group create [nome]`: Cria um grupo.
 - `/group add [user]`: Adiciona o membro.
 - `/group remove [user]`: Remove membro.
 - Dados armazenados em `groups.txt`.

Transferência de Ficheiros

- **Protocolo Inicial:** Sugeria uso de TCP para expansão futura (`file_transfer`).
- **Projeto Final:**
 - Coordenação via UDP (MSG_FILE_OFFER).
 - Envio real via TCP em porta aleatória (20000-21000).
 - **Exemplo de comando:** `/sendfile [ficheiro]`

Controlo de Inatividade

- **Protocolo Inicial:** Mencionava ping/pong opcional.
- **Projeto Final:**
 - Servidor desconecta clientes após CLIENT_TIMEOUT (60s).
 - Uso de `select()` para monitorização.

Fiabilidade UDP

- **Protocolo Inicial:** Não abordava perdas de mensagens.
- **Projeto Final:**
 - Números de sequência (SEQ:mensagem) para detetar mensagens perdidas.
 - Contadores no servidor:
 - `msgs_from_client`: Mensagens recebidas.
 - `lost_msgs_from_client`: Mensagens perdidas.
 - Exemplo: Mensagem "5:Olá" → número de sequência 5.

Comandos Adicionais

- **Mensagens Privadas:** `/dm [user]`.
- **Chat Global:** `/all`.

Avaliação Crítica:

- Comunicação em tempo real letra-a-letra.
- Modularidade e extensibilidade.
- Todos os extras implementados (grupos, ficheiros).
- Falta de encriptação (senhas/ficheiros).
- Interface do cliente pouco intuitiva.
- Suporte limitado a um grupo por cliente.

Escolhas e Descrição das implementações

1. Autenticação de Clientes-

Autenticação implementada via mensagens **MSG_AUTH** (login) e **MSG_CREATE** (criação de conta).

Os dados são guardados no ficheiro **contas.txt** (servidor).

No cliente, o utilizador é solicitado a escolher entre criar conta ou fazer login (**autenticar_ou_criar()**).

No servidor, funções como **autenticar()** e **criar_conta()** fazem verificação e escrita no ficheiro.

Os Clientes poderão enviar mensagens no chat global utilizando o comando **/all** e também poderão enviar mensagens privadas utilizando o comando: **/dm (UserName do Cliente)**.

Código de exemplo

```
94 bool criar_conta(const char *username, const char *password)
95 {
96     if (!file_exists(CONTAS_FILE))
97     {
98         FILE *f = fopen(CONTAS_FILE, "w");
99         if (!f)
100             return false;
101         fclose(f);
102     }
103     // Verifica se já existe
104     FILE *f = fopen(CONTAS_FILE, "r");
105     char u[50], p[50];
106     while (fscanf(f, "Is Is", u, p) == 2)
107     {
108         if (strcmp(u, username) == 0)
109         {
110             fclose(f);
111             return false; // Já existe
112         }
113     }
114     fclose(f);
115     // Adiciona nova conta
116     f = fopen(CONTAS_FILE, "a");
117     if (!f)
118         return false;
119     fprintf(f, "%s %s\n", username, password);
120     fclose(f);
121     return true;
122 }
```

Avaliação Crítica:

- Simples e funcional.
- Sem hashing de passwords (pouco seguros).
- Mensagens privadas e chat global enviado letra-a-letra.

2. Grupos de Clientes-

Clientes podem criar, entrar, remover membros ou apagar grupos através de comandos como:

/group create (Nome do Grupo), (Cria um Grupo e é direcionado para o mesmo)

/group add (UserName do Cliente), (Adiciona um Cliente ao Grupo)

/group remove (UserName do Cliente), (Remove um Cliente ao Grupo)

/group delete, (Apaga o grupo atual)

Grupos são guardados no ficheiro `groups.txt`.

Cada grupo tem um proprietário, membros e buffers de mensagens.

A comunicação em grupo usa o campo `group` da `Message`.



Código de exemplo

```
1  else if (strncmp(cmd, "/group create ", 14) == 0)
2  {
3      // Cria grupo
4      Message msg = {.type = MSG_GROUPS_CREATE};
5      strcpy(msg.from, username);
6      strcpy(msg.group, cmd + 14); // Nome do grupo
7      msg.data[0] = '\0';
8      sendto(sockfd, &msg, sizeof(Message), 0, (struct sockaddr *)&servaddr, sizeof(servaddr));
9      snprintf(notification_buffer, sizeof(notification_buffer), "\n* Pedido para criar grupo enviado\n");
10 }
11 else if (strncmp(cmd, "/group add ", 11) == 0)
12 {
13     // Adiciona membro ao grupo atual
14     if (strlen(current_group) == 0)
15     {
16         snprintf(notification_buffer, sizeof(notification_buffer), "\n* Selecione primeiro um grupo com /group (nome)\n");
17         return;
18     }
19     Message msg = {.type = MSG_GROUPS_ADD};
20     strcpy(msg.from, username);
21     strcpy(msg.group, current_group);
22     strcpy(msg.to, cmd + 11);
23     msg.data[0] = '\0';
24     sendto(sockfd, &msg, sizeof(Message), 0, (struct sockaddr *)&servaddr, sizeof(servaddr));
25     snprintf(notification_buffer, sizeof(notification_buffer), "\n* Pedido para adicionar membro enviado\n");
26 }
27 else if (strncmp(cmd, "/group remove ", 14) == 0)
28 {
29     // Remove membro do grupo atual
30     if (strlen(current_group) == 0)
31     {
32         snprintf(notification_buffer, sizeof(notification_buffer), "\n* Selecione primeiro um grupo com /group (nome)\n");
33         return;
34     }
35     Message msg = {.type = MSG_GROUPS_REMOVE};
36     strcpy(msg.from, username);
37     strcpy(msg.group, current_group);
38     strcpy(msg.to, cmd + 14);
39     msg.data[0] = '\0';
40     sendto(sockfd, &msg, sizeof(Message), 0, (struct sockaddr *)&servaddr, sizeof(servaddr));
41 }
42 else if (strncmp(cmd, "/group delete", 13) == 0)
43 {
44     // Apaga grupo
45     const char *group_name = NULL;
46     if (strlen(cmd) > 14) // "/group delete " + nome
47         group_name = cmd + 14;
48     else if (strlen(current_group) > 0)
49         group_name = current_group;
50     else
51         group_name = NULL;
52 }
```

Avaliação Crítica:

- O proprietário tem controle total sobre o grupo.
- Boa integração com a interface do cliente.
- Melhoria futura: permitir múltiplos grupos simultâneos em diferentes tabs no cliente.

3. Transferência de Ficheiros-

Processo híbrido: coordenação por UDP, envio real por TCP.

Cliente envia `/sendfile (ficheiro)` → cria socket TCP numa porta aleatória e avisa via `MSG_FILE_OFFER`.

Destinatários aceitam e fazem `connect()` à porta do remetente para receberem o ficheiro.

Código de exemplo:

```
1 // Comando para enviar ficheiro
2 void process_sendfile_command(const char *cmd)
3 {
4     char filename[256], target[50] = "all";
5     if (sscanf(cmd, "/sendfile %5s %s", filename, target) < 1)
6     {
7         snprintf(notification_buffer, sizeof(notification_buffer), "\n* Use: /sendfile <ficheiro> [destino]\n");
8         return;
9     }
10    FILE *fp = fopen(filename, "rb");
11    if (!fp)
12    {
13        snprintf(notification_buffer, sizeof(notification_buffer), "\n* Ficheiro não encontrado\n");
14        return;
15    }
16    fseek(fp, 0, SEEK_END);
17    size_t filesize = ftell(fp);
18    fclose(fp);
19    // Escolhe porta aleatória entre 20000-21000
20    int port = 20000 + rand() % 1000;
21    // Envia aviso UDP
22    Message msg = {.type = MSG_FILE_OFFER};
23    strcpy(msg.from, username);
24    strcpy(msg.to, target);
25    snprintf(msg.data, sizeof(msg.data), "%s|%d|%zu", filename, port, filesize);
26    sendto(sockfd, &msg, sizeof(Message), 0, (struct sockaddr *)&servaddr, sizeof(servaddr));
27    // Inicia thread servidor TCP
28    pthread_t t;
29    struct file_send_args
30    {
31        char filename[256];
32        int port;
33    } *args = malloc(sizeof(struct file_send_args));
34    strncpy(args->filename, filename, 255);
35    args->port = port;
36    pthread_create(&t, NULL, file_send_server, args);
37    pthread_detach(t);
38    snprintf(notification_buffer, sizeof(notification_buffer), "\n* Oferta de ficheiro enviada para %s\n", target);
39 }
```

Avaliação Crítica:

- Solução elegante: usa TCP para fiabilidade sem comprometer o UDP.
- Oferece controle ao destinatário (pode aceitar ou rejeitar).
- Limitação: sem encriptação ou verificação de integridade.

4. Inatividade-

O servidor monitoriza a inatividade dos clientes e remove-os após `CLIENT_TIMEOUT` (60s).

Clientes notificam o servidor ao sair (mensagem `type 3`) e recebem notificações sobre entradas/saídas.

`select()` usado para timeout e verificação periódica.

Código de exemplo:

Avaliação Crítica:

- Essencial para garantir que o estado do sistema não fique inconsistente.
- Feedback visual ao utilizador é imediato.

5. Controlo de Mensagens Recebidas e Perdidas no Servidor UDP

Estruturas de Dados

O servidor mantém, para cada cliente, os seguintes arrays globais:

- `msgs_from_client[MAX_CLIENTS]`: Conta o número total de mensagens recebidas de cada cliente.
- `lost_msgs_from_client[MAX_CLIENTS]`: Conta o número total de mensagens perdidas (não recebidas) de cada cliente.
- `last_seq_from_client[MAX_CLIENTS]`: Guarda o último número de sequência recebido de cada cliente (inicializado a -1).

Estas estruturas permitem associar o estado de cada cliente ao respetivo índice no array de clientes ativos.

Processamento de Mensagens

Ao receber uma mensagem, o servidor executa os seguintes passos:

-Identificação do Cliente

O servidor procura o cliente pelo endereço de origem (`cliaddr`). Se o cliente já existe, atualiza o campo `last_activity` e obtém o seu índice (`client_idx`).

-Extração e Verificação do Número de Sequência

Se o cliente é conhecido, o servidor extrai o número de sequência do início do campo `msg.data` (espera-se o formato `"SEQ:mensagem..."`).

Exemplo: `"5:Olá mundo"` → número de sequência 5.

-Detecção de Mensagens Perdidas

- Se já existe um número de sequência anterior (`last_seq_from_client[client_idx] != -1`) e o número de sequência atual não é o imediatamente seguinte (`seq != last_seq_from_client[client_idx] + 1`), então houve mensagens perdidas.

- O número de mensagens perdidas é calculado por:

```
lost_msgs_from_client[client_idx] += (seq - last_seq_from_client[client_idx] - 1);
```

Por exemplo, se a última mensagem recebida foi a 1 e a atual é a 3, então perdeu a 2 ($3 - 1 - 1 = 1$).

- O último número de sequência recebido é atualizado.

-Contabilização de Mensagens Recebidas

O contador `msgs\_from\_client[client\_idx]` é incrementado para cada mensagem recebida.

-Visualização do Estado

Periodicamente (a cada 10 segundos), o servidor imprime no terminal o estado de cada cliente:

=====

-----**CHECKPOINT**-----

Cliente: <username> | Mensagens recebidas: <N> | Perdas: <M>

=====

Isto permite ao operador do servidor monitorizar em tempo real a fiabilidade da comunicação UDP para cada cliente.

Conclusão

A aplicação cumpre os objetivos principais propostos, com uma interface funcional em tempo real, robustez na gestão de utilizadores e grupos, e suporte a transferência de ficheiros. As escolhas de implementação foram pragmáticas, equilibrando simplicidade e funcionalidade.

Pontos Fortes:

- Comunicação letra-a-letra em tempo real
- Modularidade e separação clara entre cliente e servidor
- Extensibilidade do protocolo
- Extras todos implementados

Pontos a Melhorar:

- Segurança (autenticação sem hash, transferência sem encriptação)
- Interface mais intuitiva no cliente
- Suporte a mais de uma conversa de grupo em paralelo